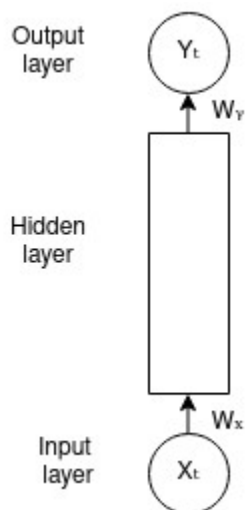


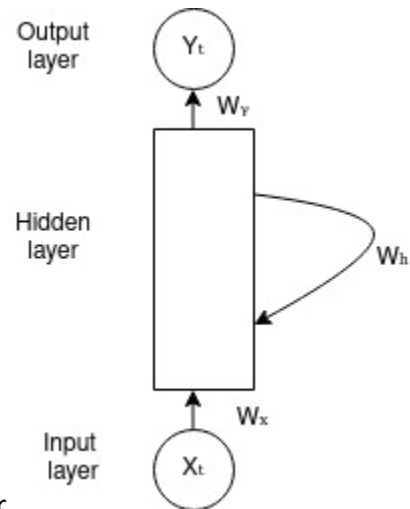
Recurrent Neural Network (RNN) Architecture

This article will provide insights into RNNs and the concept of backpropagation through time in RNN, as well as delve into the problem of vanishing and exploding gradient descent in RNNs.

Recurrent Neural Networks (RNNs) were introduced to address the limitations of traditional neural networks, such as FeedForward Neural Networks (FNNs), when it comes to processing sequential data. FNN takes inputs and process each input independently through a number of hidden layers without considering the order and context of other inputs. Due to which it is unable to handle sequential data effectively and capture the dependencies between inputs. As a result, FNNs are not well-suited for sequential processing tasks such as, language modeling, machine translation, speech recognition, time series analysis, and many other applications that requires sequential processing. To address the limitations posed by traditional neural networks, RNN comes into the picture.

RNN overcome these limitations by introducing a recurrent connection that allow information to flow from one time-step to the next. This recurrent connection enables RNNs to maintain internal memory, where the output of each step is fed back as an input to the next step, allowing the network to capture the information from previous steps and utilize it in the current step, enabling model to learn temporal dependencies and handle input of variable length.





1: FeedForward Neural Network (FNN). Image by Author

fig 2: Recurrent Neural Network (RNN). Image by Author

Architecture Of RNN

For more clear understanding of the concept of RNN, let's look at the unfolded RNN diagram.

Press enter or click to view image in full size

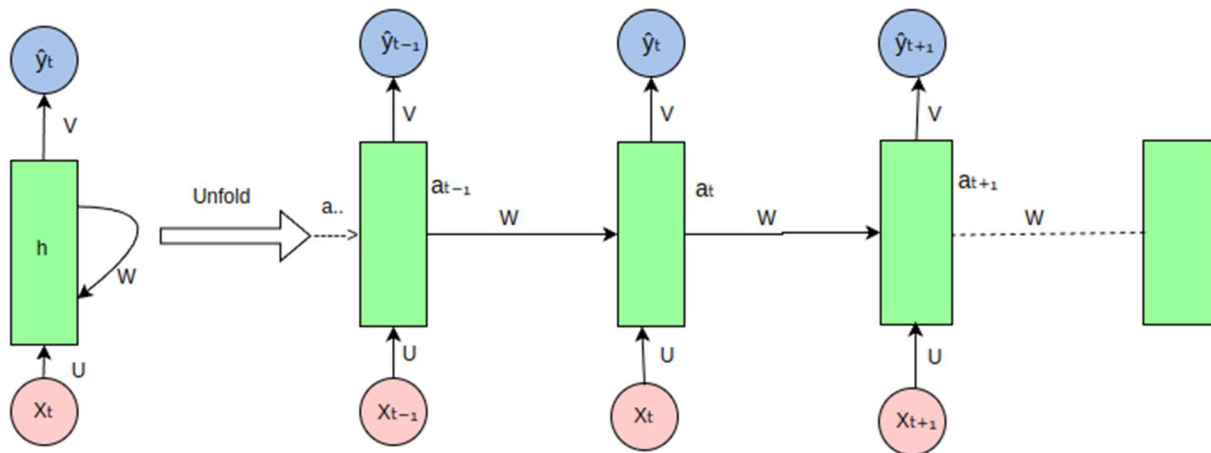


fig 3: RNN Unfolded. Image by Author.

The RNN takes an input vector $\mathbf{X}$ and the network generates an output vector $\mathbf{y}$ by scanning the data sequentially from left to right, with each time step updating the hidden state and producing an output. It shares the same parameters across all time steps. This means that, the same set of parameters, represented by U , V , W is used consistently throughout the network. U represents the weight parameter governing the connection from input layer $\mathbf{X}$ to the hidden layer $\mathbf{h}$, W represents the weight associated with the connection between hidden layers, and V for the connection from hidden layer $\mathbf{h}$ to output layer $\mathbf{y}$. This sharing of parameters allows the RNN to effectively capture temporal dependencies and process

sequential data more efficiently by retaining the information from previous input in its current hidden state.

At each time step t , the hidden state a_t is computed based on the current input x_t , previous hidden state a_{t-1} and model parameters as illustrated by the following formula:

$$a_t = f(a_{t-1}, x_t; \theta) \text{ — — — — — (1)}$$

It can also be written as,

$$a_t = f(U * X_t + W * a_{t-1} + b)$$

where,

- a_t represents the output generated from the hidden layer at time step t .
- x_t is the input at time step t .
- θ represents a set of learnable parameters(weights and biases).
- U is the weight matrix governing the connections from the input to the hidden layer; $U \in \theta$
- W is the weight matrix governing the connections from the hidden layer to itself (recurrent connections); $W \in \theta$
- V represents the weight associated with connection between hidden layer and output layer; $V \in \theta$
- a_{t-1} is the output from hidden layer at time $t-1$.
- b is the bias vector for the hidden layer; $b \in \theta$
- f is the activation function.

For a finite number of time steps $T=4$, we can expand the computation graph of a Recurrent Neural Network, illustrated in Figure 3, by applying the equation (1) $T-1$ times.

$$a_4 = f(a_3, x_4; \theta) \text{ — — — — — (2)}$$

Equation (2) can be expanded as,

$$a_4 = f(U * X_4 + W * a_3 + b)$$

$$a_3 = f(U * X_3 + W * a_2 + b)$$

$$a_2 = f(U * X_2 + W * a_1 + b)$$

The output at each time step t , denoted as y_t is computed based on the hidden state output a_t using the following formula,

$$\hat{y}_t = f(a_t; \theta) \text{ — — — — — (3)}$$

Equation (3) can be written as,

$$\hat{y}_t = f(V * a_t + c)$$

$$\text{when } t=4, \hat{y}_4 = f(V * a_4 + c)$$

where,

- $\hat{y}_t$ is the output predicted at time step t .
- V is the weight matrix governing the connections from the hidden layer to the output layer.
- c is the bias vector for the output layer.

Backpropagation Through Time (BPTT)

Backpropagation involves adjusting the model's parameters (weights and biases) based on the error between predicted output and the actual target value. The goal of backpropagation is to improve the model's performance by minimize the loss function. Backpropagation Through Time is a special variant of backpropagation used to train RNNs, where the error is propagated backward through time until the initial time step $t=1$. Backpropagation involves two key steps: forward pass and backward pass.

1. **Forward Pass:** During forward pass, the RNN processes the input sequence through time, from $t=1$ to $t=n$, where n is the length of input sequence. In each forward propagation, the following calculation takes place

$$a_t = U * X_t + W * a_{t-1} + b$$

$$a_t = \tanh(a_t)$$

$$\hat{y}_t = \text{softmax}(V * a_t + c)$$

After processing the entire sequence, RNN generates a sequence of predicted outputs $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_t]$. Loss is then computed by comparing predicted output $\hat{y}$ at each time step with actual target output y . Loss function given by,

$$L(y, \hat{y}) = (1/t) * \sum (y_t - \hat{y}_t)^2 \text{ --- --> MSE Loss}$$

2. Backward Pass: The backward pass in BPTT involves computing the gradients of the loss function with respect to the network's parameters (U, W, V and biases) over each time step.

Let's explore the concept of backpropagation through time by computing the gradients of loss at time step $t=4$. The figure below also serves as an illustration of backpropagation for time step 4.

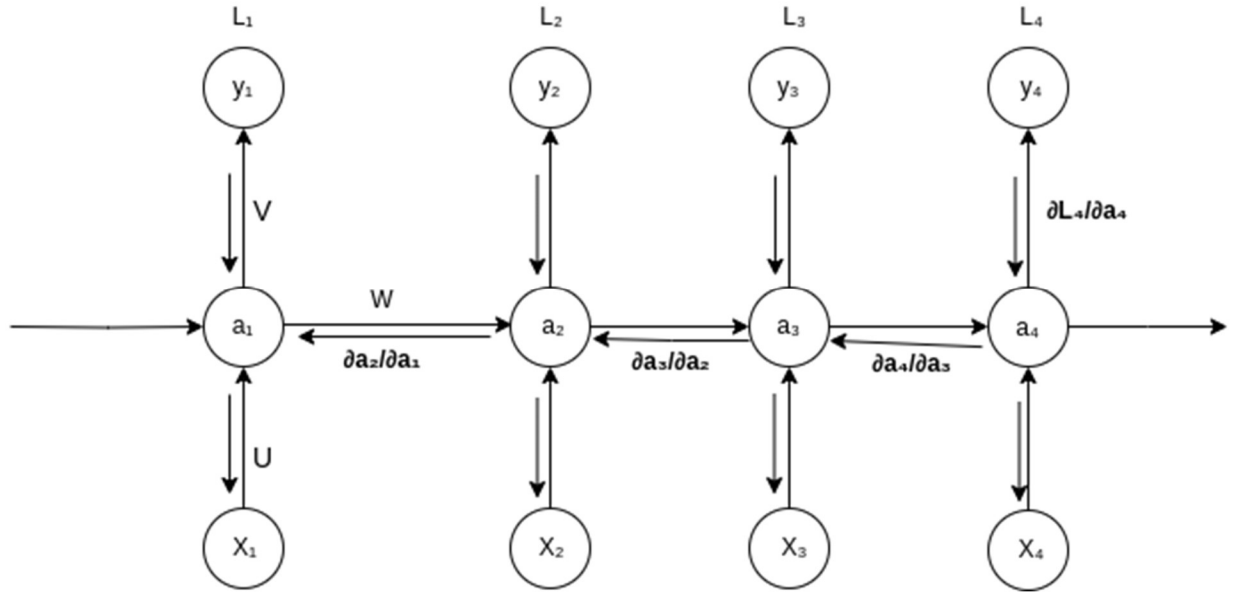


fig 4: Back Propagation Through Time (BPTT). Image by Author

Derivative of loss L w.r.t V

Loss L is a function of predicted value $\hat{y}$, so using the chain rule $\partial L / \partial V$ can be written as,

$$\partial L / \partial V = (\partial L / \partial \hat{y}) * (\partial \hat{y} / \partial V)$$

Derivative of loss L w.r.t W

Applying the chain rule of derivatives $\partial L / \partial W$ can be written as follows: The loss at the 4th time step is dependent upon $\hat{y}$ due to the fact that the loss is calculated as a function of $\hat{y}$, which is in turn dependent on the current time step's hidden state a_4 , a_4 is influenced by both W and a_3 , and again a_3 is connected to both a_2 and W, and a_2 depends on a_1 and also on W.

$$\partial L_4 / \partial W = (\partial L_4 / \partial \hat{y}_4 * \partial \hat{y}_4 / \partial a_4 * \partial a_4 / \partial W) + (\partial L_4 / \partial \hat{y}_4 * \partial \hat{y}_4 / \partial a_4 * \partial a_4 / \partial a_3 * \partial a_3 / \partial W) + (\partial L_4 / \partial \hat{y}_4 * \partial \hat{y}_4 / \partial a_4 * \partial a_4 / \partial a_3 * \partial a_3 / \partial a_2 * \partial a_2 / \partial W) + (\partial L_4 / \partial \hat{y}_4 * \partial \hat{y}_4 / \partial a_4 * \partial a_4 / \partial a_3 * \partial a_3 / \partial a_2 * \partial a_2 / \partial a_1 * \partial a_1 / \partial W)$$

Derivative of loss L w.r.t U

Similarly, $\partial L / \partial U$ can be written as,

$$\frac{\partial L_4}{\partial U} = \left(\frac{\partial L_4}{\partial \hat{y}_4} * \frac{\partial \hat{y}_4}{\partial a_4} * \frac{\partial a_4}{\partial U} \right) + \left(\frac{\partial L_4}{\partial \hat{y}_4} * \frac{\partial \hat{y}_4}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial U} \right) + \left(\frac{\partial L_4}{\partial \hat{y}_4} * \frac{\partial \hat{y}_4}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial U} \right) + \left(\frac{\partial L_4}{\partial \hat{y}_4} * \frac{\partial \hat{y}_4}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial a_1} * \frac{\partial a_1}{\partial U} \right)$$

Here we're summing up the gradients of loss across all time steps which represents the key difference between BPTT and regular backpropagation approach.

Limitations of RNN

During backpropagation, gradients can become too small, leading to the vanishing gradient problem, or too large, resulting in the exploding gradient problem as they propagate backward through time. In the case of vanishing gradients, the issue is that the gradient may become too small where the network struggles to capture long-term dependencies effectively. It can still converge during training but it may take a very very long time. In contrast, in exploding gradient problem, large gradient can lead to numerical instability during training, causing the model to deviate from the optimal solution and making it difficult for the network to converge to global minima.

Press enter or click to view image in full size

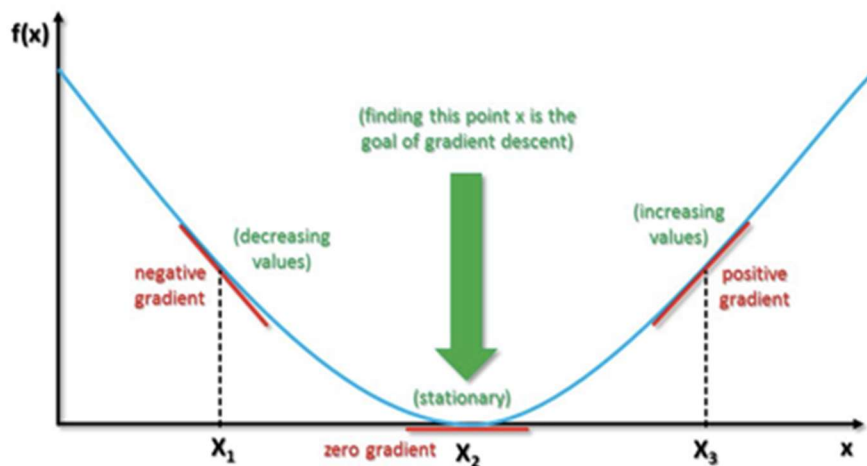
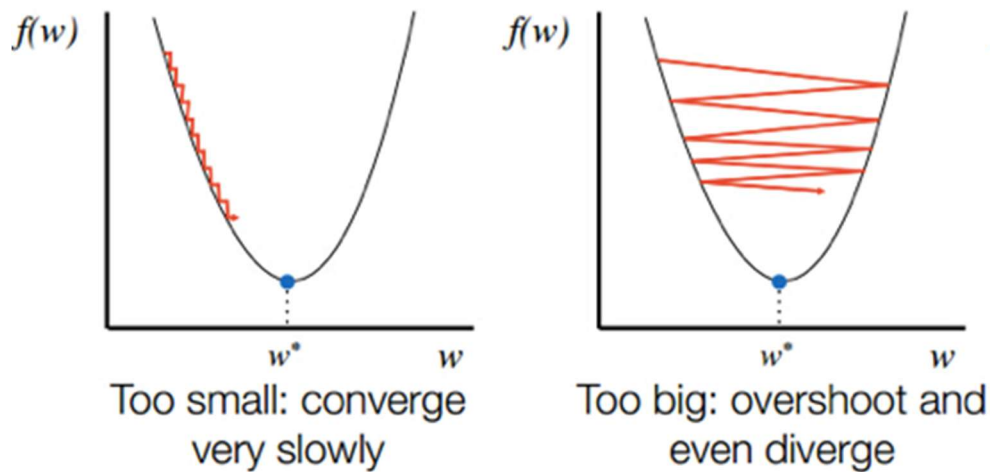


fig 5: Gradient Descent. [Image Source](#)



To address these problems, variations of RNN like Long-short term memory (LSTM) and Gated Recurrent Unit (GRU) networks have been introduced.

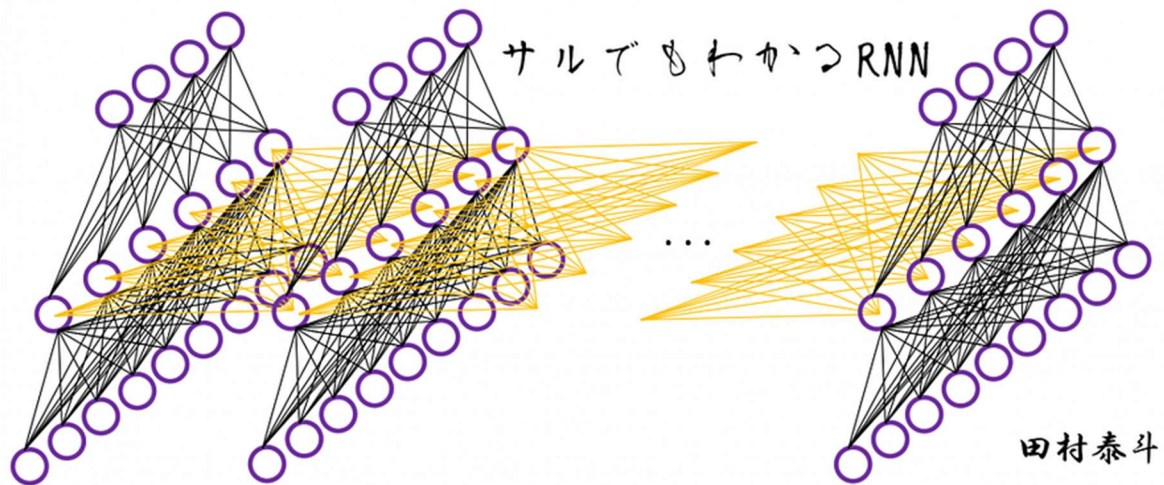
Why LSTM and GRU Introduced

4. Core Limitation of Plain RNN

Problem	Cause
Vanishing gradients	Repeated multiplication of small values
Exploding gradients	Repeated multiplication of large values
Short memory	No control over what to keep/forget
Poor long-term learning	Hidden state overwritten every step

👉 Plain RNN has no mechanism to protect important information.

Understanding LSTM: Architecture, Pros and Cons, and Implementation



[Source](#)

What is LSTM and How it works?

LSTM stands for Long Short-Term Memory, and it is a type of recurrent neural network (RNN) architecture that is commonly used in natural language processing, speech recognition, and other sequence modeling tasks.

Unlike a traditional RNN, which has a simple structure of input, hidden state, and output, an LSTM has a more complex structure with additional memory cells and gates that allow it to selectively remember or forget information from previous time steps.

An LSTM cell consists of several components:

1. **Input gate:** This gate controls the flow of information from the current input and the previous hidden state into the memory cell.
2. **Forget gate:** This gate controls the flow of information from the previous memory cell to the current memory cell. It allows the LSTM to selectively forget or remember information from previous time steps.
3. **Memory cell:** This is the internal state of the LSTM. It stores information that can be selectively modified by the input and forget gates.
4. **Output gate:** This gate controls the flow of information from the memory cell to the current hidden state and output.

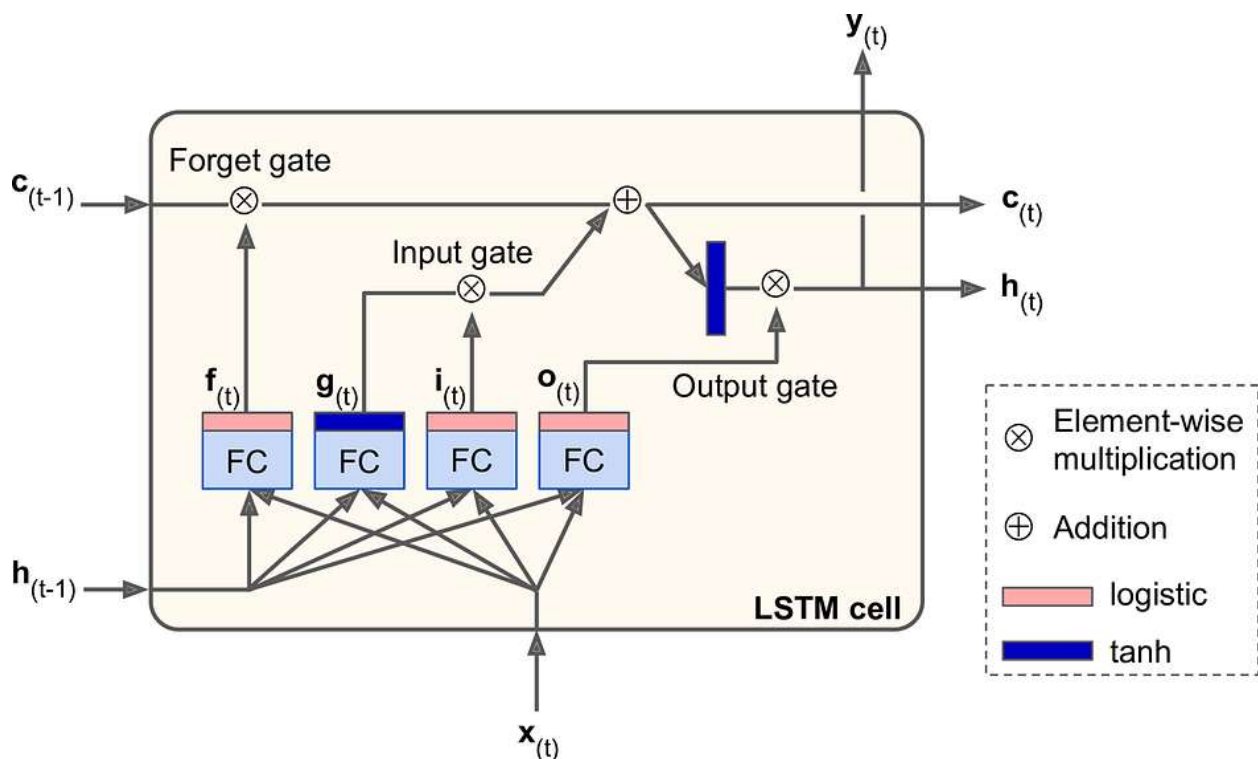
During the forward pass, the LSTM takes in a sequence of inputs and updates its memory cell and hidden state at each time step. The input gate and forget gate use sigmoid functions to

decide how much information to let into or out of the memory cell, while the output gate uses a sigmoid function and a tanh function to produce the current hidden state and output.

The LSTM's ability to selectively remember or forget information from previous time steps makes it well-suited for tasks that require modeling long-term dependencies, such as language translation or sentiment analysis.

LSTM Architecture

Press enter or click to view image in full size



[Source](#)

The architecture of an LSTM can be visualized as a series of repeating “blocks” or “cells”, each of which contains a set of interconnected nodes. Here’s a high-level overview of the architecture:

1. **Input:** At each time step, the LSTM takes in an input vector, x_t , which represents the current observation or token in the sequence.
2. **Hidden State:** The LSTM maintains a hidden state vector, h_t , which represents the current “memory” of the network. The hidden state is initialized to a vector of zeros at the beginning of the sequence.

3. **Cell State:** The LSTM also maintains a cell state vector, c_t , which is responsible for storing long-term information over the course of the sequence. The cell state is initialized to a vector of zeros at the beginning of the sequence.
4. **Gates:** The LSTM uses three types of gates to control the flow of information through the network:

a). Forget Gate: This gate takes in the previous hidden state, h_{t-1} , and the current input, x_t , and outputs a vector of values between 0 and 1 that represent how much of the previous cell state to “forget” and how much to retain. This gate allows the LSTM to selectively “erase” or “remember” information from the previous time step.

b). Input Gate: This gate takes in the previous hidden state, h_{t-1} , and the current input, x_t , and outputs a vector of values between 0 and 1 that represent how much of the current input to add to the cell state. This gate allows the LSTM to selectively “add” or “discard” new information to the cell state.

c). Output Gate: This gate takes in the previous hidden state, h_{t-1} , and the current input, x_t , and the current cell state, c_t , and outputs a vector of values between 0 and 1 that represent how much of the current cell state to output as the current hidden state, h_t . This gate allows the LSTM to selectively “focus” or “ignore” certain parts of the cell state when computing the output.

5. **Output:** At each time step, the LSTM outputs a vector, y_t , that represents the network’s prediction or encoding of the current input.

The combination of the cell state, hidden state, and gates allows the LSTM to selectively “remember” or “forget” information over time, making it well-suited for tasks that require modeling long-term dependencies or sequences.

Equations of each gates

Here are the equations for each of the three gates in an LSTM:

1. Forget Gate:

The forget gate takes as input the previous hidden state, h_{t-1} , and the current input, x_t , and outputs a vector of values between 0 and 1 that represent how much of the previous cell state to “forget” and how much to retain. The equation for the forget gate is:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

where:

σ is the sigmoid function

W_f is the weight matrix for the forget gate

$[h_{t-1}, x_t]$ is the concatenation of the previous hidden state and the current input

b_f is the bias vector for the forget gate

f_t is the vector of forget gate values for the current time step

2. Input Gate:

The input gate takes as input the previous hidden state, h_{t-1} , and the current input, x_t , and outputs a vector of values between 0 and 1 that represent how much of the current input to add to the cell state. The equation for the input gate is:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

where:

σ is the sigmoid function

W_i and W_c are the weight matrices for the input gate

$[h_{t-1}, x_t]$ is the concatenation of the previous hidden state and the current input

b_i and b_c are the bias vectors for the input gate

i_t is the vector of input gate values for the current time step

$\tilde{C}_t$ is the candidate cell state vector for the current time step, which is produced by applying the $\tanh$ activation function to a linear combination of the previous hidden state and the current input.

3. Output Gate:

The output gate takes as input the previous hidden state, h_{t-1} , the current input, x_t , and the current cell state, c_t , and outputs a vector of values between 0 and 1 that represent how much of the current cell state to output as the current hidden state, h_t . The equation for the output gate is:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad h_t = o_t * \tanh(c_t)$$

where:

σ is the sigmoid function

W_o is the weight matrix for the output gate

$[h_{t-1}, x_t]$ is the concatenation of the previous hidden state and the current input

b_o is the bias vector for the output gate

o_t is the vector of output gate values for the current time step

h_t is the current hidden state, which is produced by applying the tanh activation function to the current cell state and multiplying it element-wise with the output gate values.

Python Implementation

Here is a simple implementation of LSTM in Python using the Keras library:

```
from keras.models import Sequential
from keras.layers import LSTM, Dense

# define the LSTM model
model = Sequential()
model.add(LSTM(100, input_shape=(timesteps, features)))
model.add(Dense(1, activation='sigmoid'))

# compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# fit the model to the training data
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))
```

In this example, we first import the necessary modules from Keras. We then define the LSTM model using the Sequential API. The LSTM layer is added using the LSTM function, which takes as input the number of units (100 in this case) and the input shape (a tuple of (timesteps, features)). We then add a dense output layer with a sigmoid activation function.

We then compile the model using the compile function, specifying the loss function (binary crossentropy), optimizer (Adam), and metrics (accuracy). Finally, we fit the model to the training data using the fit function, specifying the number of epochs, batch size, and validation data.

Note that this is just a simple example, and there are many variations and customization options for LSTM models in Keras.

Pros and Cons of using LSTM

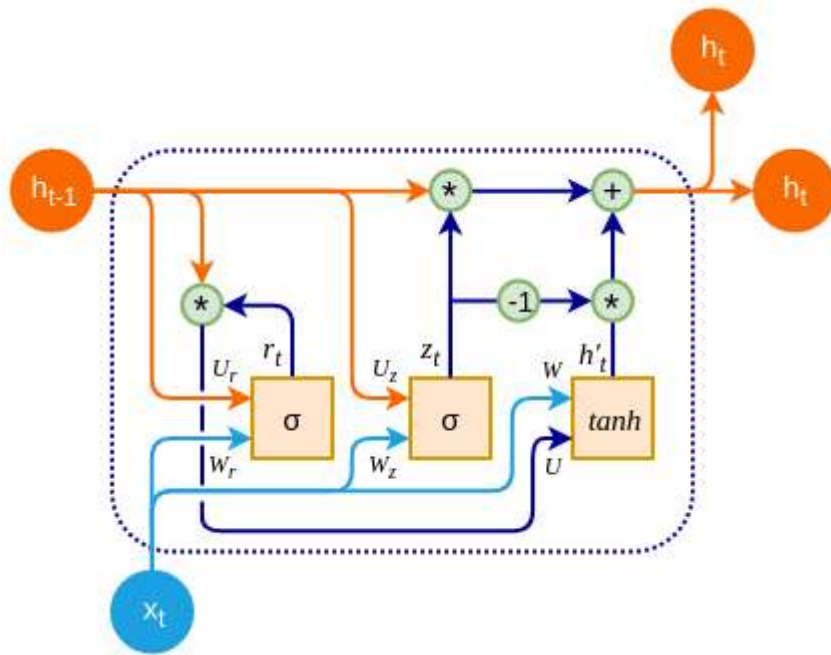
Pros:

1. **Modeling long-term dependencies:** LSTMs are well-suited for modeling long-term dependencies in sequential data, since they can selectively “remember” or “forget” information over time. This makes them useful for tasks like speech recognition, machine translation, and text generation.
2. **Robustness to noisy data:** LSTMs are more robust to noisy or missing data than other types of recurrent neural networks, since they can selectively filter out irrelevant or noisy information using the forget gate.
3. **Flexibility:** LSTMs can be used for a wide variety of tasks, including classification, regression, and sequence-to-sequence learning.
4. **Interpretability:** Since LSTMs maintain a cell state vector that represents the network’s “memory” at each time step, they can be more interpretable than other types of recurrent neural networks.

Cons:

1. **Computationally expensive:** LSTMs can be computationally expensive to train and evaluate, especially for long sequences or large datasets.
2. **Prone to overfitting:** LSTMs can be prone to overfitting on small datasets, especially if the model architecture is too complex.
3. **Hyperparameter tuning:** LSTMs have many hyperparameters that need to be tuned in order to achieve optimal performance, including the number of hidden units, the learning rate, and the dropout rate.
4. **Data requirements:** LSTMs require a large amount of training data to learn complex patterns in the data. If there is not enough data available, the model may not perform well.

Understanding Gated Recurrent Unit (GRU) in Deep Learning



What is Gated Recurrent Unit (GRU) ?

GRU stands for Gated Recurrent Unit, which is a type of recurrent neural network (RNN) architecture that is similar to LSTM (Long Short-Term Memory).

Like LSTM, GRU is designed to model sequential data by allowing information to be selectively remembered or forgotten over time. However, GRU has a simpler architecture than LSTM, with fewer parameters, which can make it easier to train and more computationally efficient.

The main difference between GRU and LSTM is the way they handle the memory cell state. In LSTM, the memory cell state is maintained separately from the hidden state and is updated using three gates: the input gate, output gate, and forget gate. In GRU, the memory cell state is replaced with a "candidate activation vector," which is updated using two gates: the reset gate and update gate.

The reset gate determines how much of the previous hidden state to forget, while the update gate determines how much of the candidate activation vector to incorporate into the new hidden state.

Overall, GRU is a popular alternative to LSTM for modeling sequential data, especially in cases where computational resources are limited or where a simpler architecture is desired.

How GRU Works?

Like other recurrent neural network architectures, GRU processes sequential data one element at a time, updating its hidden state based on the current input and the previous hidden state. At

each time step, the GRU computes a “candidate activation vector” that combines information from the input and the previous hidden state. This candidate vector is then used to update the hidden state for the next time step.

The candidate activation vector is computed using two gates: the reset gate and the update gate. The reset gate determines how much of the previous hidden state to forget, while the update gate determines how much of the candidate activation vector to incorporate into the new hidden state.

Here’s the math behind the GRU architecture:

1. The reset gate r and update gate z are computed using the current input x and the previous hidden state h_{t-1}

$$r_t = \text{sigmoid}(W_r * [h_{t-1}, x_t])$$

$$z_t = \text{sigmoid}(W_z * [h_{t-1}, x_t])$$

where W_r and W_z are weight matrices that are learned during training.

2. The candidate activation vector $h_t^{\sim}$ is computed using the current input x and a modified version of the previous hidden state that is "reset" by the reset gate:

$$h_t^{\sim} = \tanh(W_h * [r_t * h_{t-1}, x_t])$$

where W_h is another weight matrix.

3. The new hidden state h_t is computed by combining the candidate activation vector with the previous hidden state, weighted by the update gate:

$$h_t = (1 - z_t) * h_{t-1} + z_t * h_t^{\sim}$$

Overall, the reset gate determines how much of the previous hidden state to remember or forget, while the update gate determines how much of the candidate activation vector to incorporate into the new hidden state. The result is a compact architecture that is able to selectively update its hidden state based on the input and previous hidden state, without the need for a separate memory cell state like in LSTM.

GRU Architecture

The GRU architecture consists of the following components:

1. **Input layer:** The input layer takes in sequential data, such as a sequence of words or a time series of values, and feeds it into the GRU.
2. **Hidden layer:** The hidden layer is where the recurrent computation occurs. At each time step, the hidden state is updated based on the current input and the previous hidden

state. The hidden state is a vector of numbers that represents the network's "memory" of the previous inputs.

3. **Reset gate:** The reset gate determines how much of the previous hidden state to forget. It takes as input the previous hidden state and the current input, and produces a vector of numbers between 0 and 1 that controls the degree to which the previous hidden state is "reset" at the current time step.
4. **Update gate:** The update gate determines how much of the candidate activation vector to incorporate into the new hidden state. It takes as input the previous hidden state and the current input, and produces a vector of numbers between 0 and 1 that controls the degree to which the candidate activation vector is incorporated into the new hidden state.
5. **Candidate activation vector:** The candidate activation vector is a modified version of the previous hidden state that is "reset" by the reset gate and combined with the current input. It is computed using a tanh activation function that squashes its output between -1 and 1.
6. **Output layer:** The output layer takes the final hidden state as input and produces the network's output. This could be a single number, a sequence of numbers, or a probability distribution over classes, depending on the task at hand.

Pros and Cons of GRU

Like any machine learning model, Gated Recurrent Unit (GRU) neural networks have both advantages and disadvantages. Here are some pros and cons of using GRU:

Pros:

- GRU networks are similar to Long Short-Term Memory (LSTM) networks, but with fewer parameters, making them computationally less expensive and faster to train.
- GRU networks can handle long-term dependencies in sequential data by selectively remembering and forgetting previous inputs.
- GRU networks have been shown to perform well on a variety of tasks, including natural language processing, speech recognition, and music generation.
- GRU networks can be used for both sequence-to-sequence and sequence classification tasks.

Cons:

- GRU networks may not perform as well as LSTMs on tasks that require modeling very long-term dependencies or complex sequential patterns.
- GRU networks may be more prone to overfitting than LSTMs, especially on smaller datasets.
- GRU networks require careful tuning of hyperparameters, such as the number of hidden units and learning rate, to achieve good performance.
- GRU networks may not be as interpretable as other machine learning models, since the gating mechanism can make it difficult to understand how the network is making predictions.

Overall, GRU networks are a powerful tool for modeling sequential data, especially in cases where computational resources are limited or where a simpler architecture is desired. However, like any machine learning model, they have their limitations and require careful consideration when choosing the appropriate model for a particular task.